

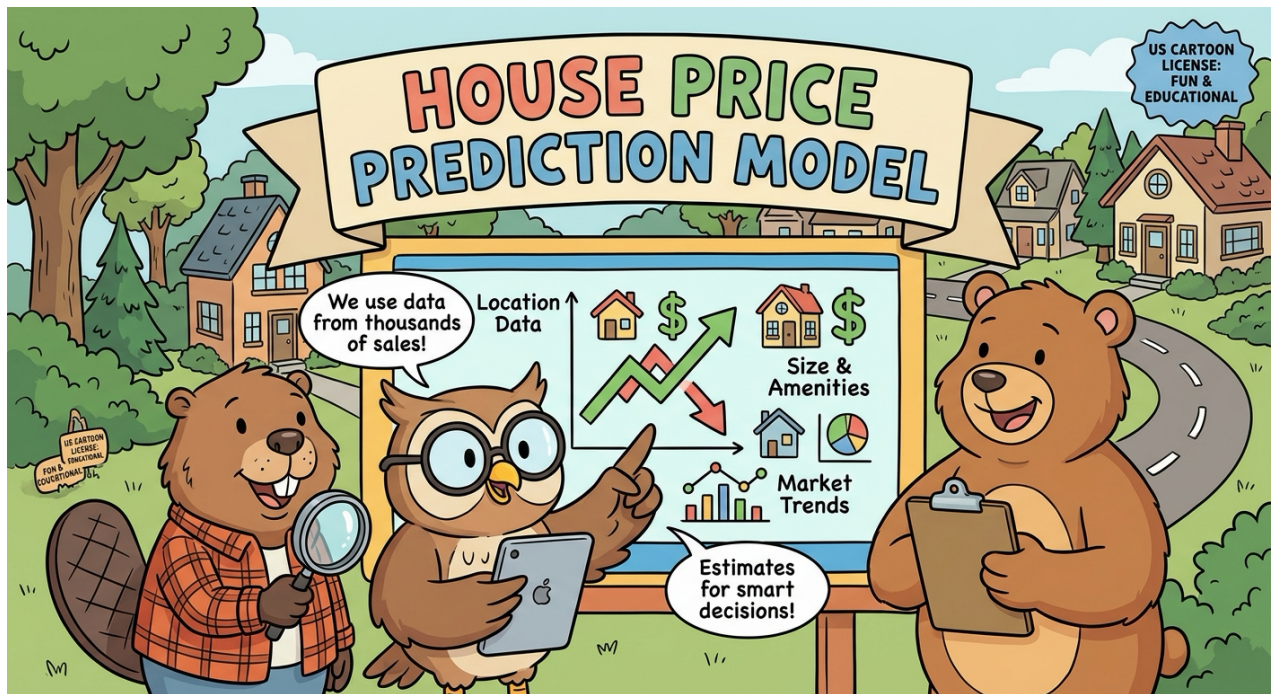
CP020003 — Artificial Intelligence 2026

Khon Kaen University

Week 10— Homework Assignment

In class this week you built a full regression ladder to price a consumer loan — plain Linear Regression, regularized linear models (Ridge, Lasso, ElasticNet), tree ensembles (Random Forest, Gradient Boosting, XGBoost, LightGBM, CatBoost), a PyTorch MLP, a modern pretrained tabular foundation model (TabPFN), and a GenAI layer that translates the model's own feature importances into plain English — all benchmarked with MAE, RMSE, MAPE, R^2 , and MedAE against a “just guess the average” baseline.

This assignment is deliberately open-ended: you will use the exact same techniques, but on your own dataset — a Kaggle-style house price dataset from Khon Kaen University's course repository — with your own train/validation split and your own combination of model families, to investigate a pricing question you find genuinely interesting. There is no single “correct” model to land on — you are graded on how rigorously you engineer features, fit, diagnose, and compare, and how convincingly you turn the numbers into a business recommendation.



1. Dataset

Use the KKU house price dataset from the course GitHub repository below (a single CSV file, already cleaned — every column is numeric, so there is no string-parsing step this time):

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1/blob/main/dataset/kaggle_kku_house_price_dataset.csv

The CSV has one row per house, with the following columns:

Column	Meaning
Square_Footage	Total finished living area of the house, in square feet.
Num_Bedrooms	Number of bedrooms.
Num_Bathrooms	Number of bathrooms.
Year_Built	Year the house was constructed. Consider engineering $\text{House_Age} = \text{current year} - \text{Year_Built}$.
Lot_Size	Size of the land parcel (acres).
Garage_Size	Number of cars the garage holds (0, 1, or 2).
Neighborhood_Quality	An index from 1 (lowest) to 10 (highest) rating the desirability of the neighborhood.
House_Price	Sale price of the house, in USD — this is your regression target.

Tip: decide up front whether you will model `House_Price` directly or a transformed target such as $\log(\text{House_Price})$, and say why. House prices are typically right-skewed (a few very expensive homes), so a log transform can make a linear model's errors more evenly spread across the price range — but it also changes how you must interpret RMSE/MAE (in log-dollars, not dollars) and requires you to exponentiate predictions back before reporting a dollar figure.

2. Your Task 🎯

Suggested Investigation Path (guided warm-up)

Before your open-ended investigation, work through these five guided questions from the in-class notebook — they check that your mechanics (train/validation split, preprocessing pipeline, feature diagnostics, and scoring) are solid before you go explore on your own:

1. Change the validation split size. Try `test_size` of 0.1, 0.2, and 0.3 instead of the class default. How does the baseline's ("predict the training mean") validation RMSE change as the validation set grows? Plot RMSE vs. `test_size` for the baseline to show the effect directly.
2. Compute the correlation of every numeric feature with `House_Price`, exactly as in the leakage-check section of class. Unlike the loan dataset, none of these columns look temporally suspicious — but pick the single strongest correlate and, in one sentence, explain why it makes economic sense that this feature would dominate a home's price.
3. Fit Linear Regression, Ridge, Lasso, and ElasticNet on the same preprocessing pipeline (as in Section 9 of class). Print and compare their coefficients — does Lasso zero out any feature entirely, and if so, does that match your intuition about which features carry the least signal?
4. Tune one hyperparameter each for Random Forest and XGBoost (e.g. `max_depth` or `n_estimators`). Try at least three settings for each and plot validation RMSE vs. the hyperparameter value. Does more trees / more depth keep helping, or does it plateau or overfit on a dataset this size?
5. (Challenge) Fit the PyTorch MLP from Section 11 of class on this dataset and compare its validation RMSE against your best tree ensemble. Give a one-sentence hypothesis for why a small MLP does or does not beat gradient boosting on a small, mostly-linear tabular dataset like this one.

Open-Ended Investigation (the main graded task)

This is where most of your grade comes from. Design a pricing study around this house price dataset using the same ladder as class — linear, tree ensembles, and deep learning (with the modern foundation-model and GenAI layers as bonus extensions) — but with choices that are your own:

- Pick your target and framing. Raw House_Price or a log-transformed target? Justify the choice using what you saw in the EDA step below.
- Justify your feature set. Which raw columns did you keep, and which engineered features (e.g. House_Age, price per square foot as a diagnostic — not as a training feature, since it would leak the target, Bedrooms-to-Bathrooms ratio) did you add, and why do you believe each one carries signal rather than noise?
- Decide what “worth deploying” means for your investigation. Lowest RMSE in dollars? Lowest MAPE (fairest across cheap and expensive homes alike)? Most interpretable coefficients for a real-estate agent to explain to a client? State your criterion up front, before you start comparing models.
- This is the same kind of design decision our Week 9 assignment asked you to make when choosing a forecast horizon and evaluation criterion for TSLA — here, the design decision is which pricing objective matters most when every row is a one-off asset (a house) rather than a repeated time series observation. Document your reasoning.

Required Steps

(mirrors our in-class notebook)

1. EDA & cleaning — load the CSV, check for missing values and obvious outliers, and plot the distribution of House_Price plus scatter plots of House_Price against Square_Footage and Neighborhood_Quality, as in Section 5 of class.
2. Feature engineering & sanity check — engineer at least one new feature (e.g. House_Age from Year_Built) and confirm, the way class checked Amount.Funded.By.Investors, that none of your engineered features accidentally encode the target itself.
3. Train/validation split — hold out a validation set with train_test_split (state your test_size and random_state) and build one ColumnTransformer/Pipeline that every model reuses, exactly as in Section 6 of class, so the comparison stays fair.
4. Pick and justify ≥ 2 model families from: Linear (Linear Regression/Ridge/Lasso/ElasticNet), Tree Ensembles (Random Forest/Gradient Boosting/XGBoost/LightGBM/CatBoost), and Deep Learning (PyTorch MLP). Briefly explain, referencing the recap table from class, why each family is a reasonable fit for a small, mostly-numeric housing dataset like this one.
5. Fit and predict with each chosen family on the same train/validation split and the same feature set — no feature may be derived using information that would not exist at prediction time.
6. Score every model against the baseline (“predict the training-set mean price”) using MAE, RMSE, MAPE, and R^2 , exactly as in Section 7–8 of class, and note whether each model actually beats the baseline.
7. Interpretability — for at least one linear model, report its coefficients; for at least one tree model, run permutation importance and, if time allows, a SHAP summary plot (Section 14 of class). Which features actually drive predicted price?
8. Partial dependence — plot how predicted price moves as Square_Footage changes, holding everything else fixed (Section 15 of class). Does the relationship look linear, or does it flatten/bend at the extremes?
9. Business interpretation — in plain language, state which model or family you would trust to price a new listing and why, backed by your actual MAE/RMSE/MAPE numbers and what a typical error means in dollars to a homeowner or agent.
10. Conclusion — which family won for your chosen target framing, what trade-off did you accept to get there (e.g. interpretable coefficients vs. raw accuracy, or training time vs. accuracy), and which model would you actually put in front of a real-estate agent first, and why?

Use this table as a starting skeleton for your final scoreboard (feel free to add rows/columns your investigation needs):

Model	Family	MAE	RMSE	MAPE (%)	R ²	Beats baseline?
Baseline (mean)	Baseline					—
	Linear					
	Tree Ensemble					
	Deep Learning					

⚠ Grading Emphasis — Read This Carefully

Getting one model to run is necessary, but it is not sufficient to score well this week. Most of the marks for this assignment are earned in Steps 6–10 above: how rigorously you benchmark your ≥ 2 model families against the baseline on the same validation set, and how clearly you can explain, in plain language backed by your own MAE/RMSE/MAPE numbers, which model you would actually trust to price a house.

A submission that fits one linear model and one tree ensemble, but checks feature importance carefully, engineers one thoughtful new feature, and backs its recommendation with real numbers, will score higher than a submission that runs five architectures but only offers a single vague sentence of comparison at the end.

3. Bonus Points ✨

- Add a third model family beyond your required two (e.g. bring in the PyTorch MLP if your two were Linear + Tree Ensemble) and extend your scoreboard — +bonus
- Try TabPFN, the pretrained tabular foundation model from Section 12 of class, on this dataset and compare it against your best tree ensemble — does a zero-training-time foundation model keep up on a dataset this small? — +bonus
- Add the GenAI explanation layer from Section 13 of class: take your best model's top feature importances and generate a short plain-English explanation of what drives a house's predicted price — +bonus
- Fit your best model on both raw House_Price and $\log(\text{House_Price})$ and report exactly how much the log transform changes RMSE/MAPE and the shape of your residuals — +bonus
- Combine your pricing model with a simple decision rule: flag any house in your validation set where the actual price is more than X% below the model's predicted price as a potential “good deal,” and discuss how you would sanity-check that flag before acting on it — +bonus